Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Telekomunikacji

Praca dyplomowa magisterska

na kierunku Inżynieria Internetu Rzeczy

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie działające modele LLM na platformie sprzętowej o ograniczonych zasobach.

Krzysztof Kulma
Numer albumu 318476

promotor
dr inż. Daniel Paczesny

WARSZAWA 2026

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie działające modele LLM na platformie sprzętowej o ograniczonych zasobach.

Streszczenie. Celem pracy było zwiększenie interaktywności wczasowiczów przebywających w domkach wypoczynkowych Ośrodka Wypoczynkowego Politechniki Warszawskiej w Wildze. Zrealizowano prototyp w postaci lokalnego asystenta głosowego, który umożliwia wczasowiczom dostęp do informacji zbieranych przez sieć czujników IoT. Asystent ma w przystępny sposób tłumaczyć odczyty dotyczące warunków w domku i zużycia energii, bez konieczności korzystania z interfejsu technicznego.

Część implementacyjna polegała na zaprojektowaniu i uruchomieniu kompletnego systemu do interakcji głosowej. Obejmuje on detekcję mowy, transkrypcję, dynamiczne budowanie kontekstu z danych IoT, lokalną generację odpowiedzi oraz syntezę mowy. System został oparty na architekturze dwumodelowej. Większy model „Nauczyciel" asynchronicznie analizuje dane z sieci czujników, a mniejszy model „Uczeń" odpowiada na bieżąco na pytania użytkownika na urządzeniu brzegowym.

Część badawcza polegała na empirycznym doborze małego modelu językowego oraz parametrów jego pracy na urządzeniu o ograniczonych zasobach. Na Raspberry Pi 5 porównano kilka małych modeli językowych z rodzin Gemma, Llama, Qwen, Phi i Bielik, pochodzących od różnych dostawców. Zbadano wpływ kwantyzacji wag, hiperparametrów generacji oraz konstrukcji promptu na jakość odpowiedzi w języku polskim i na czas ich uzyskania.

Wykonana implementacja i przeprowadzone testy na Raspberry Pi 5 pozwoliły wykazać, że użyteczny asystent głosowy może działać lokalnie, bez wysyłania zapytań do chmury. Najbardziej efektywna zbadana konfiguracja, model `gemma3:4b` poddany kwantyzacji Q4_K_M, z ponownym użyciem stałego prefiksu promptu, osiągnęła medianę czasu odpowiedzi (E2E, od zapytania do ostatniego tokenu) poniżej 10 s przy wskaźniku halucynacji 1,17%. Otrzymane wyniki wskazują, że o jakości odpowiedzi decyduje przede wszystkim wybór modelu, a skrócenie opóźnienia wymaga utrzymania modelu w pamięci urządzenia i obniżenia precyzji wag.

Słowa kluczowe: asystent głosowy, LLM, IoT, smart home, inferencja lokalna

Design and implementation of a voice assistant using locally running LLM models on a resource-constrained hardware platform.

Abstract. The aim of this thesis was to increase the interactivity of guests staying at the Warsaw University of Technology holiday cottages in Wilga. A prototype was implemented in the form of a local voice assistant that gives guests access to information collected by an IoT sensor network. The assistant explains cottage conditions and energy use in everyday language, so there is no need for a technical interface.

The implementation work designed and ran a complete voice interaction system. It covers speech detection, transcription, dynamic context building from IoT data, local response generation, and speech synthesis. The system was based on a dual-model architecture. A larger "Teacher" model analyses data from the sensor network in the background, and a smaller "Student" model answers the user in real time on the edge device.

The research work selected a small language model and its operating parameters empirically, on a resource-constrained device. Several small language models from the Gemma, Llama, Qwen, Phi and Bielik families, from different vendors, were compared on a Raspberry Pi 5. The study examined how weight quantization, generation hyperparameters, and prompt design affect the quality of answers in Polish and the time needed to produce them.

The implementation and the tests on a Raspberry Pi 5 showed that a useful voice assistant can run locally, without sending queries to the cloud. The most effective configuration studied, the `gemma3:4b` model quantized to Q4_K_M, with reuse of a fixed prompt prefix, reached a median end-to-end response time, from query to last token, below 10 s, at a hallucination rate of 1.17%. The obtained results indicate that answer quality depends primarily on the choice of model, while reducing latency requires keeping the model in device memory and lowering the precision of its weights.

Keywords: voice assistant, LLM, IoT, smart home, on-device inference

Spis treści

1. Wstęp . . . 10
1.1. Tło problemu . . . 10
1.2. Teza badawcza i cel pracy . . . 10
1.3. Zakres pracy . . . 11
2. Przegląd literatury i analiza rozwiązań . . . 12
2.1. Małe modele językowe . . . 12
2.1.1. Modele językowe . . . 12
2.1.2. Wielomodelowe systemy LLM . . . 13
2.1.3. Inżynieria promptów i dostarczanie kontekstu . . . 14
2.1.4. Optymalizacja hiperparametrów generacji . . . 15
2.1.5. Fine tuning modeli językowych . . . 16
2.2. Inferencja na urządzeniu brzegowym . . . 16
2.2.1. Architektury hybrydowe Edge–Cloud . . . 16
2.2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU . . . 17
2.3. Interfejs głosowy . . . 18
2.3.1. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy . . . 18
3. Koncepcja i architektura systemu . . . 20
3.1. Założenia projektowe . . . 20
3.2. Architektura Nauczyciel–Uczeń . . . 20
3.3. Schemat przepływu danych . . . 20
4. Środowisko projektowo-badawcze . . . 21
4.1. Wybór platformy brzegowej . . . 21
4.2. Architektura sprzętowa . . . 21
4.3. Modele LLM w przetwarzaniu brzegowym danych . . . 21
4.4. Wykorzystane oprogramowanie i technologie . . . 21
4.4.1. Aplikacja użytkownika . . . 21
4.4.2. Warstwa usług . . . 21
4.4.3. Moduły STT i TTS . . . 21
4.4.4. LLM . . . 21
4.4.5. Moduł kontekstu . . . 21
4.4.6. Narzędzia badawcze . . . 22
5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu . . . 23
5.1. Architektura i metodologia systemu testowego . . . 23
5.1.1. Komponenty systemu testowego . . . 23
5.1.2. Zbiór danych testowych . . . 23
5.1.3. Metryki oceny . . . 23
5.1.4. Dostrajanie parametrów pracy modelu . . . 23
5.1.5. Porównywane modele . . . 23
5.2. Etap Optuna . . . 23

5.2.1. Warunek termiczny pomiaru . . . 23
5.2.2. Dominacja flagi halucynacji . . . 23
5.2.3. Opóźnienie E2E i model Gaussa . . . 24
5.3. Etap kwantyzacji . . . 24
5.3.1. Typy kwantyzacji GGUF . . . 24
5.3.2. Wstępne pomiary E2E . . . 24
5.3.3. Wyniki . . . 24
5.3.4. Anomalie i wnioski . . . 24
5.4. Inżynieria promptów i buforowanie prefiksu . . . 24
5.4.1. Buforowanie prefiksu promptu systemowego . . . 24
5.4.2. Budowa promptu systemowego . . . 24
5.5. Rekomendacje dla prototypu . . . 24
6. Część implementacyjna: realizacja prototypu . . . 25
6.1. Architektura prototypu . . . 25
6.2. Elementy systemu i wykorzystane technologie . . . 25
6.2.1. Aplikacja użytkownika . . . 25
6.2.2. Transkrypcja mowy (STT) . . . 25
6.2.3. Moduł kontekstu . . . 25
6.2.4. LLM Nauczyciel . . . 25
6.2.5. SLM Uczeń . . . 25
6.2.6. Synteza mowy (TTS) . . . 25
6.2.7. Konfiguracja . . . 25
6.3. Testy prototypu . . . 26
7. Podsumowanie . . . 27
7.1. Wnioski z przeprowadzonych badań . . . 27
7.2. Ocena prototypu względem założeń . . . 27
7.3. Plan rozwoju . . . 27
Bibliografia . . . 29

Wersja dla promotora z 18.10.2026. Do redakcji włączono streszczenia, rozdział 1 (wstęp) oraz rozdział 2 (przegląd literatury). Kolejne rozdziały pozostają poza zakresem tej wersji.

1. Wstęp

1.1. Tło problemu

W ramach projektu Wilga MiLL Uczelnia prowadziła badania w domkach wczasowych na terenie Ośrodka Wypoczynkowego Politechniki Warszawskiej w Wildze. W domkach testowano system IoT monitorujący zachowania użytkowników. Celem projektu było zbudowanie inteligentnej przestrzeni edukacyjnej typu *living lab* nakierowanej na edukację prośrodowiskową. W ramach badań w jednym z domków zaimplementowano sieć czujnikową IoT. W niniejszej pracy wykorzystano zebrane przez nią dane archiwalne.

System czujników generował znaczne ilości surowych danych. Choć były one interesujące z perspektywy badawczej, pozostawały trudne do interpretacji dla przeciętnej osoby odwiedzającej Ośrodek. Nadmiar informacji oraz brak intuicyjnego interfejsu prowadzi do niepełnego wykorzystania potencjału systemu. Utrudniało to realizację celu projektu, jakim było zwiększenie świadomości zużycia energii i wpływu człowieka na środowisko. Problem braku zainteresowania systemem IoT wśród wczasowiczów był już wcześniej przedmiotem badań zespołu projektowego. W ramach projektu INOT opracowano aplikację komunikującą się poprzez interfejs wizualny. Aplikacja wprowadziła postać Wilgusia, ptaka gospodarza domu. Wilguś miał przedstawiać dane w zrozumiały sposób i zwiększać atrakcyjność działania systemu.

Istotnym elementem tła problemu były również zbadane zachowania użytkowników. Dotychczasowe doświadczenia zespołu projektowego wskazywały na ambiwalentny stosunek wczasowiczów do obecności technologii w domkach. Niektórzy postrzegali urządzenia IoT jako zbędne lub naruszające ich prywatność. Przejawiało się to np. poprzez odłączanie urządzeń od zasilania, co powodowało utrudnienia w działaniu systemu. Doświadczenia te wskazywały na potrzebę zwiększenia atrakcyjności działania systemu. Wczasowicze powinni mieć poczucie aktywnego korzystania z niego, a nie bycia obserwowanymi przez czujniki IoT.

1.2. Teza badawcza i cel pracy

Przyjęto założenie, że istniejący interfejs wizualny należy rozbudować o interfejs głosowy. Interakcja z nim powinna być naturalna, płynna i intuicyjna. Nie powinna być dla wczasowicza obciążeniem, lecz atrakcyjnym elementem pobytu w Wildze. System powinien w subtelny sposób dostarczać informacji interesujących także dla osoby nietechnicznej. Miało to zachęcać wczasowiczów do interakcji z inteligentnym otoczeniem zamiast skłaniać ich do wyłączania urządzeń IoT.

Postawiono tezę, że na urządzeniach o ograniczonych zasobach można uruchomić mały model językowy (SLM), stanowiący rdzeń responsywnego asystenta głosowego korzystającego z danych systemu IoT. Asystent powinien komunikować się w języku polskim, a interakcja głosowa przebiegać płynnie. Założenie to można zrealizować z wykorzystaniem koncepcji współpracy modeli „Nauczyciel”–„Uczeń”, w której bardziej złożone zadania są

delegowane do „nauczyciela", czyli modelu działającego poza urządzeniem brzegowym, natomiast odpowiedzi kierowane do użytkownika generowane są lokalnie.

Głównym celem pracy było opracowanie, wdrożenie i przetestowanie asystenta głosowego, który ułatwia wczasowiczom zrozumienie danych zbieranych przez system IoT. Cele cząstkowe obejmowały zaprojektowanie architektury systemu, dobór i optymalizację małego modelu językowego do pracy na urządzeniu o ograniczonych zasobach, opracowanie mechanizmu dynamicznego budowania kontekstu, integrację komponentów w spójny system interakcji głosowej, a także ocenę jakości i czasu odpowiedzi.

Realizacja celu wymagała badań obejmujących m.in. dobór modelu i parametrów generacji, kwantyzację wag, a także ocenę jakości odpowiedzi i czasu działania na urządzeniu o ograniczonych zasobach.

1.3. Zakres pracy

Prace rozpoczęto od etapu koncepcyjnego. Najpierw poszerzono wiedzę w obszarach związanych z tematem pracy. W rozdziale 2 przedstawiony został przegląd literatury dotyczącej architektur brzegowych i chmurowych, kwantyzacji modeli, systemów wielomodelowych, inżynierii promptów, optymalizacji parametrów generacji oraz interfejsów głosowych.

Na tej podstawie opracowano koncepcję architektury systemu oraz sposobu wdrożenia komponentów. W rozdziale 3 przedstawiono podział zadań między modułami systemu oraz przepływ danych między nimi. Następnie przygotowano środowisko projektowo-badawcze. W niniejszym rozdziale opisano również wybór platformy Raspberry Pi 5 jako reprezentanta urządzeń o ograniczonych zasobach, a także architekturę sprzętową oraz wykorzystane oprogramowanie otwartoźródłowe.

Fazę badawczą opisano w rozdziale 5. Obejmowała ona dobór i optymalizację konfiguracji modelu językowego uruchamianego na platformie o ograniczonych zasobach. Analizie poddano wpływ parametrów uruchomieniowych, wielkości modelu, kwantyzacji wag, zawartości promptu oraz warunków pracy urządzenia na jakość i czas generowania odpowiedzi.

Część implementacyjna objęła wdrożenie asystenta głosowego zgodnie z opracowaną koncepcją oraz rekomendacjami wynikającymi z badań. Opis tej fazy przedstawiono w rozdziale 6. Prace skupiły się na zbudowaniu modułów systemu, a następnie ich integracji. Zbudowano w ten sposób spójny przepływ danych: od zapytania głosowego użytkownika, przez transkrypcję i budowanie kontekstu, po lokalną inferencję modelu Ucznia i syntezę odpowiedzi. Ponieważ podczas prac dane z Wilgi nie były dostępne na żywo, przygotowano warstwę danych zbudowaną w oparciu o dane archiwalne z okresu około 90 dni.

2. Przegląd literatury i analiza rozwiązań

Pracę nad rozwiązaniem poprzedzono przeglądem literatury oraz dostępnych rozwiązań. Przetwarzanie języka naturalnego (ang. *Natural Language Processing*, NLP) jest dziś zdominowane przez modele językowe (ang. *Large Language Models*, LLM). Sam rynek LLM jest jednak bardzo szeroki. Jego główną część stanowią trenowane wielkie modele językowe o dziesiątkach i setkach miliardów parametrów, budowanie systemów w oparciu o chmurowe interfejsy programistyczne, systemy multimodalne, agenci planujący wieloetapowe zadania oraz asystenci komercyjni. Niniejszy rozdział nie obejmuje tego obszaru w całości. Przegląd zawężono do trzech zagadnień bezpośrednio związanych z tematem pracy. Omówiono małe modele językowe (ang. *Small Language Models*, SLM), inferencję na urządzeniu brzegowym oraz interfejs głosowy.

W kolejnych podrozdziałach przedstawiono najpierw małe modele językowe oraz wybrane wzorce współpracy modeli, następnie zagadnienia inferencji na urządzeniu brzegowym, a na końcu elementy interfejsu głosowego.

2.1. Małe modele językowe

Niniejszy podrozdział wprowadza pojęcie małych modeli językowych oraz przegląda rozwiązania wielomodelowe, inżynierię promptów, dobór hiperparametrów generacji i fine-tuning.

2.1.1. Modele językowe

Fundamentem przetwarzania języka naturalnego jest dziś architektura Transformer, zaproponowana przez Vaswaniego et al. [1]. W transformerze tekst przetwarzany jest jako ciąg tokenów (fragmenty słów) a zależności między nimi modelowane są wyłącznie za pomocą mechanizmu atencji (ang. *attention*). Na jego podstawie każdy token wyznacza, które z pozostałych elementów sekwencji są istotne dla jego znaczenia w danym zdaniu. W odróżnieniu od wcześniej stosowanych sieci rekurencyjnych (ang. *Recurrent Neural Networks*, RNN) Transformer nie przetwarza tekstu sekwencyjnie, analizuje równolegle. Pozwoliło to na trenowanie modeli językowych na bardzo dużych zbiorach danych. Pojedynczy blok architektury składa się mechanizmu atencji (ang. *multi-head attention*) oraz sieci jednokierunkowej (ang. *feed-forward network*), a kompletny model powstaje przez nałożenie na siebie kilkudziesięciu takich bloków.

Jedną z odmian architektury Transformer jest Mistral, wprowadzony przez Jianga et al. [2]. Jest to zoptymalizowana architektura typu *decoder-only*, wykorzystująca grupowaną atencję zapytań (ang. *Grouped-Query Attention*, GQA) oraz atencję z przesuwnym oknem (ang. *Sliding Window Attention*, SWA). Mechanizmy te ograniczają zużycie pamięci i koszt obliczeniowy, zwiększając dzięki teu wydajność inferencji. Dzięki wysokiej wydajności i otwartej licencji Apache 2.0 model Mistral 7B stał się inspiracją dla wielu nowych modelu, w tym rodziny polskich modeli językowych Bielik.

W ostatnich latach widoczny stał się nurt rozwijania małych modeli językowych (ang. *Small Language Models*, SLM), o rozmiarze rzędu kilku miliardów parametrów. Dzięki

treningowi prowadzonemu na wysokiej jakości, selektywnie dobranych danych osiągi tych modeli potrafią zbliżyć się do jednostek wielokrotnie od nich większych. Potwierdzają to raporty techniczne opublikowane przez firmy Meta [3], Microsoft [4] oraz Google [5]. Modele Llama 3, Phi-3 i Gemma 2 osiągają wyniki konkurencyjne względem modeli o rozmiarach rzędu dziesiątek miliardów parametrów, przy znacznie mniejszych wymaganiach sprzętowych. Polskie modele z rodziny Bielik również opierają się na architekturze Transformer: większe warianty (7 i 11 mld parametrów) wywodzą się z modelu Mistral 7B [6], a najmniejsze (1,5 i 4,5 mld) z modeli Qwen2.5 [7].

Rynek małych modeli językowych rozwija się dynamicznie. W trakcie realizacji badań będących częścią niniejszej pracy na rynku pojawiły się kolejne generacje modeli znanych dostawców. Badania pokazują, że nawet przy tej samej architekturze i tym samym producencie konfiguracja optymalna dla jednego modelu nie musi sprawdzać się w modelu kolejnej generacji [8].

2.1.2. Wielomodelowe systemy LLM

Literatura opisuje kilka wariantów współpracy modeli językowych o różnej wielkości. Poniżej omówiono trzy z nich, destylację wiedzy, wnioskowanie kaskadowe oraz dekodowanie spekulatywne.

Hierarchiczny podział ról między modelami jest pojęciem wywodzącym się z destylacji wiedzy (ang. *knowledge distillation*). Zostało ono wprowadzone przez Hintona et al. [9]. Destylacja opiera się na przekazywaniu wiedzy pomiędzy modelami. Wększy model, zwany "Nauczycielem"(ang. *teacher*) transferuje wiedzę do mniejszego modelu- "Ucznia"(ang. *student*). Dzieje się to poprzez udostępnianie tak zwanych miękkich etykiet (ang. *soft labels*) zamiast konkretnych stałych odpowiedzi. Podejście to wykorzystuje większe możliwości modeli o dużej liczbie parametrów, aby usprawniać modele mniejsze. Nauczyciel (większy model) dysponuje bogatszą wiedzą, a uczeń (model mniejszy) próbuje odwzorować jego zachowanie przy zdecydowanie mniejszych zasobach. Autorzy wykazali, że tak przeprowadzony proces transferu wiedzy pozwala mniejszemu modelowi podnieść jakość odpowiedzi w konkretnym zadaniu.

Innym wzorcem jest paradygmat kaskadowego wnioskowania (ang. *LLM cascading*). Chen et al. [10] pokazali, że sekwencyjne odpytywanie modeli wraz z wstrzymywaniem procesu w odpowienim momencie (uniknięcie przetrenowanie), redukuje czas inferencji nawet o 98% przy zachowaniu jakości odpowiedzi modelu większego. W tym podejściu podział ról nie jest ustalony z góry (tak jak w destylacji wiedzy), lecz wynika ze specyfiki konkretnego zadania. Proste przypadki obsługiwane są przez tańszy, mniejszy model, zatomiast zadania trudniejsze trafiają do jednostki o większych możliwościach.

Innym wzorcem współpracy modeli jest dekodowanie spekulatywne (ang. *speculative decoding*), zaproponowane przez Leviathana et al. [11]. Mały i szybki model, tzw. model szkicowy (ang. *draft model*), proponuje ciąg kolejnych tokenów (tekstu odpowiedzi), a model docelowy weryfikuje je w czasie generacji, akceptując albo odrzucając kolejne propozycje. Autorzy wykazali, że taka procedura zmniejsza czas generacji, ponieważ rola wolniejszego z modeli ograniczona jest do recenzji gotowej już odpowiedzi. Mały model

odpowiada szybciej, więc został użyty do generowania szkicu, duży zaś jest gwarantem jakości odpowiedzi.

Z przeprowadzonego przeglądu wynika, że współpraca modeli o różnej wielkości jest szeroko opisana w literaturze i obejmuje kilka różnych podejść. Destylacja wiedzy dotyczy etapu trenowania, wnioskowanie kaskadowe dynamicznego przydziału zadań, a dekodowanie spekulatywne pozwala przyspieszyć generację tokenów. Wspólną ideą pozostaje jednak hierarchia możliwości, w której mniejszy model działa na sprzęcie o ograniczonych zasobach, natomiast większy wspiera jego pracę. Celem niniejszej pracy jest uruchomienie modelu SLM na urządzeniu o ograniczonych zasobach, dlatego wzorzec nauczyciel–uczeń wymaga pewnej modyfikacji względem klasycznej destylacji. Zamiast transferu wiedzy podczas procesu trenowania, może być ona przekazywana przez większy model asynchronicznie, tak aby nie obciążać urządzenia brzegowego w momencie interakcji.

2.1.3. Inżynieria promptów i dostarczanie kontekstu

Literatura opisuje różne sposoby kształtowania zapytań kierowanych do modeli językowych oraz dostarczania im kontekstu, w którym zadawane jest pytanie. Poniżej omówiono techniki inżynierii promptów, metody kontrolowania zachowania modelu oraz podejście Retrieval-Augmented Generation (RAG).

Inżynieria promptów (ang. *prompt engineering*) zajmuje się sposobem kształtowania zapytań trafiających do modelu językowego. Ma ona na celu poprawę jakości odpowiedzi oraz kontrolowanie ich formy [12]. Schulhoff et al. dokonali analizy i klasyfikacji technik promptowania (pisania zapytań), wyróżniając między innymi podejście bez przykładów (ang. *zero-shot*), podejście z kilkoma przykładami umieszczonymi w prompcie (ang. *few-shot*) oraz podejście z narzuconymi krokami rozumowania (ang. *chain-of-thought*). Brown et al. [13] pokazali, że duże modele potrafią dostosować swoje zachowanie do zadania wyłącznie na podstawie przykładów zawartych w prompcie. Możliwe jest więc "pseudo uczenie"bez modyfikacji wag modelu. Zjawisko to określa się jako uczenie w kontekście (ang. *in-context learning*). Wei et al. [14] wykazali, że zastosowanie kolejnych kroków rozumowania (ang. *chain-of-thought*) może znacząco poprawić jakość odpowiedzi w zadaniach wymagających wieloetapowego działania, nawet bez dodatkowego trenowania modelu.

Istotnym elementem inżynierii promptów jest również kontrolowanie zachowania modelu (ang. *constraining*). Najczęściej realizowane jest ono za pomocą promptu systemowego (ang. *system prompt*), czyli jednej stałej instrukcji doklejanej do wiadomości wejściowej. Prompt sytemowy określa zasady generowania odpowiedzi, jej charakter czy sposób komunikacji z użytkownkiem. Modele językowe, w szczególności SLM (ang. *small language models*), mogą generować informacje nieobecne w dostarczonym kontekście. Zjawisko to określa się mianem halucynacji (ang. *hallucination*). Odpowiednio sformułowany prompt systemowy pozwala ograniczać takie zachowanie poprzez określenie zasad, na przykład nakazując modelowi komunikowanie braku danych zamiast generowania niepotwierdzonych informacji.

Innym powszechnie stosowanym sposobem dostarczania wiedzy do modelu jest podejście Retrieval-Augmented Generation (RAG). Zostało ono przedstawione przez Lewisa et

al. [15] jako rozwiązanie łączące wiedzę zapisaną w wagach modelu z wiedzą dostarczaną zewnętrznie poprzez wyszukiwanie odpowiednich informacji (ang. *retrieval*). W typowym rozwiązaniu RAG fragmenty dokumentów pobierane są z bazy wektorowej na podstawie podobieństwa semantycznego, a następnie dołączane do kontekstu zapytania. Pozwala to modelowi korzystać jedynie z danych istotnych w kontekście konkretnego zapytania użytkownia, co podnosi jakość odpowiedzi i zmniejsza czas generacji.

Z przeprowadzonego przeglądu wynika, że zarówno inżynieria promptów, jak i dostarczanie zewnętrznego kontekstu pozwalają wpływać na zachowanie modelu bez konieczności modyfikowania jego wag. Ma to szczególne znaczenie w przypadku modeli działających na urządzeniach o ograniczonych zasobach, gdzie dodatkowe trenowanie może być kosztowne. Z perspektywy niniejszej pracy za szczególnie interesujące uznano wykorzystanie promptu systemowego oraz dostarczanie modelowi dodatkowej wiedzy bez konieczności modyfikowania jego wag. Istotnym kierunkiem jest również wykorzystanie idei RAG do przekazywania aktualnych danych z sieci czujników IoT. W tym kontekście warto rozważyć strukturyzowane wstrzykiwanie kontekstu (ang. *structured context injection*), które pozwala dostarczać modelowi odpowiednie informacje bez konieczności stosowania klasycznego mechanizmu wyszukiwania.

2.1.4. Optymalizacja hiperparametrów generacji

Literatura opisuje wiele metod doboru parametrów generacji modeli językowych, w tym metody pozwalające ograniczyć koszt ich optymalizacji.

Parametry pracy modelu językowego, takie jak temperatura próbkowania (ang. *sampling temperature*), rozmiar okna kontekstu (ang. *context window*), maksymalna długość odpowiedzi, współczynnik top-p oraz kara za powtórzenia, mają wpływ na jakość generowanej odpowiedzi oraz czas jej generacji. Optymalne wartości parametrów nie są jednak stałe nawet dla konkretnego modelu, zależą one m.in. od zadania i sprzętu, na którym działa model. Przeszukiwanie pełnej siatki kombinacji wymienionych parametrów jest kosztowne obliczeniowo. Nawet niewielka przestrzeń parametrów może dać wiele kombinacji, co powoduje, że testowanie ich wszystkich jest czasowo nieopłacalne.

Autorzy pracy [16] przedstawiają bibliotekę Optuna, framework optymalizacji hiperparametrów umożliwiający wykorzystanie algorytmu TPE (ang. *Tree-structured Parzen Estimator*) do optymalizacji dowolnej siatki hiperparametrów. Zamiast testować kombinacje losowo lub w zadanej kolejności, algorytm TPE buduje probabilistyczny model na podstawie zebranych dotychczas wyników i kieruje kolejne próby w stronę najbardziej obiecujących konfiguracji. Metoda ta pozwala ograniczyć liczbę ewaluacji i znajduje zastosowanie w optymalizacji wieloparametrowej.

Z przeprowadzonego przeglądu wynika, że Optuna może stanowić alternatywę dla kosztownego przeszukiwania pełnej siatki parametrów. Z perspektywy niniejszej pracy interesujące jest jej wykorzystanie do doboru parametrów generacji modelu SLM działającego na urządzeniu o ograniczonych zasobach.

2.1.5. Fine tuning modeli językowych

Literatura opisuje różne metody dostrajania modeli językowych, w tym rozwiązania pozwalające ograniczyć koszt tego procesu. Poniżej omówiono metody LoRA oraz QLoRA.

W odróżnieniu od kwantyzacji, omówionej w sekcji 2.2.2, która dotyczy redukcji precyzji wag gotowego modelu, *fine-tuning* (dostrajanie) polega na dostosowaniu modelu do zadania poprzez aktualizację parametrów w procesie treningu. Pełne dostrajanie (ang. *full fine-tuning*) modeli posiadających miliardy parametrów jest kosztowne obliczeniowo, szczególnie jeśli jest przeprowadzane lokalnie.

Autorzy pracy [17] zaproponowali metodę LoRA (ang. *Low-Rank Adaptation*), która nie wymaga modyfikacji wszystkich wag modelu. Zamiast tego polega na dodaniu par niewielkich macierzy o niskim rzędzie (ang. *low-rank matrices*), których iloczyn wyznacza zmianę wag, podczas gdy wagi bazowe pozostają zamrożone. LoRA redukuje liczbę trenowalnych parametrów nawet 10 000-krotnie, przy zachowaniu jakości porównywalnej lub nawet wyższej od pełnego dostrajania dla standardowych zadaniań. Dodatkowe macierze można scalić z wagami bazowymi po zakończeniu treningu, eliminując dodatkowy narzut podczas inferencji.

Dettmers et al. [18] rozszerzyli metodę LoRA o kwantyzację wag modelu bazowego podczas trenowania. Nową metodę nazwali QLoRA. Wagi zamrożonego modelu bazowego przechowywane są w precyzji 4-bitowej, natomiast obliczenia związane z gradientami wykonywane są w wyższej precyzji dla zachowania stabilności procesu. Umożliwia to np. dostrajanie modeli o 65 miliardach parametrów na GPU wyposażonym w 48 GB pamięci VRAM. Kwantyzację wag w kontekście inferencji, a nie trenowania, omówiono w sekcji 2.2.2.

Z przeprowadzonego przeglądu wynika, że metody LoRA i QLoRA pozwalają ograniczyć wymagania sprzętowe związane z dostrajaniem modeli językowych. Z perspektywy niniejszej pracy szczególnie interesujące jest ich potencjalne wykorzystanie do adaptacji modelu SLM do konkretnego zadania, którym jest odpowaidanie na zapytania wczasowiczów w Wildze.

2.2. Inferencja na urządzeniu brzegowym

2.2.1. Architektury hybrydowe Edge–Cloud

Modele językowe mogą być uruchamiane lokalnie, na urządzeniu brzegowym (ang. *edge computing*), albo po stronie serwera, w chmurze (ang. *cloud computing*).

Architektura chmurowa oferuje bardzo duże zasoby obliczeniowe, zwykle większe niż urządzenia brzegowe czy serwery prywatne. Wadą tych rozwiązań jest niezerowe opóźnienie komunikacyjne, wynikające z konieczności transmitowania danych. Drugim ograniczeniem obliczeń chmurowych jest zależność od stałego połączenia z internetem. Istotne pozostają również kwestie prywatności. Dane użytkownika muszą zostać przesłane do zewnętrznego dostawcy mocy obliczeniowej [8].

Architektura brzegowa umożliwia przetwarzanie lokalne i eliminuje narzut komunikacyjny. Dane pozostają na urządzeniu. Powoduje to jednak pewne ograniczenia. Urządzenia brzegowe dysponują zazwyczaj znacznie mniejszą mocą obliczeniową, dlatego stosowane

modele muszą być odpowiednio mniejsze. Badania empiryczne pokazują, że procesory ARM w komputerach jednopłytkowych osiągają wydajność rzędu od 5 do 15 tokenów na sekundę dla modeli, których wielkość mieści się w granicach 1,5 miliarda parametrów oraz od 2 do 15 tokenów na sekundę dla modeli posiadających 3 miliardy parametrów [19]. Dla tej klasy sprzętu modele o rozmiarze od 2 do 4 miliardów parametrów stanowią często stosowany kompromis między jakością a czasem generacji [19].

Wąskim gardłem inferencji (procesu generowania odpowiedzi) na procesorze bez GPU pozostaje przepustowość pamięci (ang. *memory bandwidth*) oraz konieczność utrzymania temperatury poniżej progu przegrzewania [20], [21]. Generowanie kolejnych tokenów wymaga cyklicznego odczytu wag neuronów z pamięci, dlatego im większy model i dłuższy kontekst, tym bardziej skomplikowany jest proces generowania. Z literatury wynika, że na urządzeniach brzegowych skrócenie czasu odpowiedzi wymaga ograniczenia liczby wag (dobór modelu) i długości przetwarzanego kontekstu, a nie wyłącznie zwiększania mocy obliczeniowej.

Kompromisem pomiędzy architekturami brzegową i chmurową jest architektura hybrydowa, w której trudniejsze rozumowanie (ang. *reasoning*) realizowane jest na lokalnym serwerze, natomiast inferencja obsługująca użytkownika odbywa się na urządzeniu brzegowym [8]. Podejście to zachowuje prywatność danych (brak ryzyka związanego z transmisją), ponieważ serwer działa lokalnie, jednocześnie umożliwiając użycie modelu większego niż ten uruchamiany bezpośrednio na urządzeniu brzegowym. Modele na lokalnym serwerze nie są jednak zwykle tak rozbudowane jak modele dostępne w chmurze.

Z przeprowadzonego przeglądu wynika, że dla asystenta korzystającego z danych systemu IoT pełne przeniesienie inferencji do chmury jest niekorzystne. Wprowadza zależność od łącza internetowego, bardzo niestabilnego w Wildzie. Czyste przetwarzanie brzegowe z kolei wymusza użycie jedynie małego modelu językowego i ograniczenie długości kontekstu. Modele mniejsze niż około 1,5 miliarda parametrów często nie osiągają zadowalającej jakości odpowiedzi w języku polskim, natomiast modele przekraczające 4 miliardy parametrów wymagają możliwości przekraczających dostępne lokalnie moce obliczeniowe [19]. Kierunkiem przyjętym w dalszej części pracy jest więc architektura hybrydowa składająca się z lokalnego serwera wspomagającego analizę oraz urządzenia brzegowego obsługującego odpowiedź skierowaną do użytkownika.

2.2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU

Kwantyzacja jest popularną techniką redukcji precyzji wag modeli LLM w celu zmniejszenia ich wielkości i opóźnień odpowiedzi modelu. Znajduje ona swoje zastosowanie w dwóch odrębnych kontekstach: procesie trenowania (fine-tuning, sekcja 2.1.5) oraz optymalizacji samej inferencji (przyspieszenie prędkości generowania). Niniejsza podsekcja dotyczy drugiego z wymienionych zastosowań, zmniejszenia precyzji wag gotowego modelu w celu przyspieszenia generacji tokenów i redukcji zużycia zasobów platformy sprzętowej podczas inferencji.

Jednym z głównych ograniczeń występujących w procesie inferencji dużych modeli językowych jest przepustowość pamięci (ang. *memory-bandwidth-bound*), a nie wydaj-

ność obliczeniowa. Kim et al. [20] wykazali, że generowanie kolejnych tokenów wymaga odczytywania wag modelu z pamięci, wobec czego zmniejszenie ich precyzji bitowej może ograniczyć opóźnienia inferencji. Przykładowo model o wielkości 4 miliardów parametrów w precyzji FP16 (16 bitów) zajmuje ok. 8 GB pamięci. Po kwantyzacji do 4 bitów na wagę (Q4_K_M) jego rozmiar może zmniejszyć się do ok. 2,5 GB, co ogranicza ilość danych odczytywanych podczas generacji kolejnych tokenów.

Frantar et al. [22] zaproponowali metodę GPTQ (ang. *Generative Pre-Trained Transformers Quantization*). Metoda ta polega na przeprowadzeniu procesu kwantyzacji jednokrotnie, dopiero po zakończeniu trenowania (ang. *post-training quantization*, PTQ). Metoda ta pozwala skwantyzować model o wielkości 175 miliardów parametrów do 3 lub 4 bitów w około cztery godziny pracy GPU. Efektem końcowym jest model znacznie lżejszy, o niewielkim spadku jakości generowanej odpowiedzi. Innym rozwiązaniem stosowanym szeroko w świecie LLM i SLM jest format GGUF, wykorzystywany między innymi przez bibliotekę llama.cpp.

Format GGUF (ang. *GPT-Generated Unified Format*) umożliwia przechowywanie wag w tak zwanych zgrupowanych schematach kwantyzacji (ang. *groupwise quantization*), w których grupy wag mogą współdzielić współczynnik skalowania. Poszczególne schematy różnią się precyzją: Q4_K_M (4 bity) może niejednokrotnie stanowić kompromis między rozmiarem a jakością; Q8_0 (8 bitów) zapewnia wyższą precyzję kosztem większego zużycia pamięci; natomiast Q2_K i Q3_K agresywnie redukują rozmiar przy zauważalnym spadku jakości [23]. Schemat Q4_K_M stał się popularnym rozwiązaniem na sprzęcie o ograniczonych zasobach, w tym na architekturze ARM [8]. W modelach MoE (ang. *Mixture-of-Experts*) agresywna kwantyzacja do Q2_K może natomiast powodować znaczący spadek jakości. Dzieje się tak ze względu na wrażliwość poszczególnych komponentów modelu na błędy kwantyzacji.

Z przeprowadzonego przeglądu wynika, że kwantyzacja jest skuteczną metodą ograniczania zużycia pamięci i może przyspieszać generację szczególnie na urządzeniach o ograniczonych zasobach. Jest tak, ponieważ redukcja precyzji wag zmniejsza wolumen danych odczytywanych przy każdym tokenie. Na platformach brzegowych dodatkowym ograniczeniem pozostaje też utrzymanie temperatury procesora pod kontrolą [21]. Z perspektywy niniejszej pracy szczególnie interesujące jest wykorzystanie modeli skwantyzowanych w formacie GGUF oraz porównanie różnych poziomów kwantyzacji na urządzeniu o ograniczonych zasobach. Istotne jest również wsparcie tego formatu (GGUF) przez bibliotekę llama.cpp, stanowiącą domyślny silnik inferencji aplikacji Ollama.

2.3. Interfejs głosowy

2.3.1. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy

Interakcja głosowa z modelem językowym wymaga kilku mechanizmów odpowiedzialnych za przetwarzanie mowy. Poniżej omówiono detekcję aktywności głosowej (VAD), rozpoznawanie mowy (STT) oraz syntezę mowy (TTS).

Przepływ danych w systemie głosowym wymaga mechanizmu detekcji aktywności głosowej (ang. *Voice Activity Detection*, VAD), odróżniającego segmenty mowy od ciszy i szumów tła. Bez VAD model odpowiedzialny za rozpoznawanie mowy przetwarzałby strumień audio w sposób ciągły. Takie rozwiązanie byłoby nieakceptowalne, ponieważ może generować fałszywe transkrypcje z segmentów ciszy i zwiększać opóźnienia. Zastosowanie jasnego sygnału startowego (ang. *start-of-speech*) pozwala na ograniczenie niepotrzebnego przetwarzania. Silero Team [24] opracowało lekki model neuronowy VAD do wdrożeń brzegowych (ang. *edge-computing*). Przetwarza on fragmenty audio o długości około 30 ms w czasie poniżej 1 ms na jednostce CPU, przy zachowaniu niezwykle małego rozmiaru modelu (poniżej 1 MB). Model wspiera próbkowanie 8 kHz i 16 kHz, dostępny jest na licencji MIT oraz zoptymalizowany m.in. dla biblioteki PyTorch.

W rozpoznawaniu mowy (ang. *Speech-to-Text*, STT) modelem open-source będącym standardem rynkowym stał się Whisper, opracowany przez Radforda et al. [25]. Model ten został wytrenowany metodą słabego nadzoru (ang. *weak supervision*) na 680 000 godzin wielojęzycznego meteriału audio. Dzięki takiej technice uzyskano wysoką odporność na szumy tła, co ma znaczenie w środowiskach o zmiennych warunkach akustycznych. Whisper wspiera transkrypcję języka polskiego oraz jest dostępny bezpłatnie w kilku wariantach rozmiaru. Implementacja *faster-whisper* została zoptymalizowana pod kątem wydajności i może być wykorzystywana na urządzeniach o ograniczonych zasobach.

Modeli służących do syntezy mowy (ang. *Text-to-Speech*, TTS) jest dziś na rynku bardzo wiele. Obok komercyjnych platform o wysokiej bardoz jakości, takich jak ElevenLabs [26], funkcjonuje wiele rozwiązań otwartoźródłowych. Klasycznym modelem end-to-end jest m.in. VITS [27] oferujący naturalne brzmienie przy niskim opóźnieniu. Innym przykładem jest biblioteka `edge-tts` [28], która udostępnia głosy znane z przeglądarki Microsoft Edge, w tym między innymi polski głos `pl-PL-ZofiaNeural`.

Z przeprowadzonego przeglądu wynika, że istnieją rozwiązania pozwalające realizować poszczególne etapy interakcji głosowej również na urządzeniach o ograniczonych zasobach. Z perspektywy niniejszej pracy szczególnie interesujące jest wykorzystanie lekkiego mechanizmu VAD oraz modelu Whisper w implementacji *faster-whisper* do rozpoznawania mowy. W konktekście syntezy mowy istotna jest natomiast dostępność polskiego języka, którą zapewnia np. `edge-tts`. Takie podejście pozwala zbudować kompletny interfejs głosowy.

3. Koncepcja i architektura systemu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.1. Założenia projektowe

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.2. Architektura Nauczyciel–Uczeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

3.3. Schemat przepływu danych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4. Środowisko projektowo-badawcze

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.1. Wybór platformy brzegowej

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.2. Architektura sprzętowa

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.3. Modele LLM w przetwarzaniu brzegowym danych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4. Wykorzystane oprogramowanie i technologie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.1. Aplikacja użytkownika

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.2. Warstwa usług

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.3. Moduły STT i TTS

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.4. LLM

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.5. Moduł kontekstu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

4.4.6. Narzędzia badawcze

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1. Architektura i metodologia systemu testowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.1. Komponenty systemu testowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.2. Zbiór danych testowych

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.3. Metryki oceny

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.4. Dostrajanie parametrów pracy modelu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.1.5. Porównywane modele

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2. Etap Optuna

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.1. Warunek termiczny pomiaru

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.2. Dominacja flagi halucynacji

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.2.3. Opóźnienie E2E i model Gaussa

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3. Etap kwantyzacji

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.1. Typy kwantyzacji GGUF

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.2. Wstępne pomiary E2E

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.3. Wyniki

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.3.4. Anomalie i wnioski

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4. Inżynieria promptów i buforowanie prefiksu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4.1. Buforowanie prefiksu promptu systemowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.4.2. Budowa promptu systemowego

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

5.5. Rekomendacje dla prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6. Część implementacyjna: realizacja prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.1. Architektura prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2. Elementy systemu i wykorzystane technologie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.1. Aplikacja użytkownika

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.2. Transkrypcja mowy (STT)

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.3. Moduł kontekstu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.4. LLM Nauczyciel

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.5. SLM Uczeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.6. Synteza mowy (TTS)

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.2.7. Konfiguracja

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

6.3. Testy prototypu

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7. Podsumowanie

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.1. Wnioski z przeprowadzonych badań

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.2. Ocena prototypu względem założeń

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

7.3. Plan rozwoju

Ten fragment został usunięty z bieżącej wersji, ponieważ powstał przed pierwszą rundą uwag.

Bibliografia

[1] A. Vaswani i in., "Attention is all you need", *Advances in neural information processing systems*, t. 30, 2017, Seminal paper on Transformer architecture.

[2] A. Q. Jiang i in., "Mistral 7B", *arXiv preprint arXiv:2310.06825*, 2023. DOI: 10.48550/arXiv.2310.06825

[3] A. Dubey, A. Jauhri, A. Pandey, A. Keshwam, A. Al-Dahle i in., "The Llama 3 Herd of Models", *Meta Publications*, 2024, Meta AI Technical Report. URL: https://ai.meta.com/research/publications/the-llama-3-herd-of-models/

[4] M. Abdin, S. A. Jacobs, A. A. Awan, J. Bi i in., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone", *arXiv preprint arXiv:2404.14219*, 2024, Microsoft Research. URL: https://arxiv.org/abs/2404.14219

[5] G. Team i G. DeepMind, "Gemma 2: Improving Open Language Models at a Practical Size", *Semantic Scholar*, 2024, Google DeepMind Technical Report. URL: https://www.semanticscholar.org/paper/Gemma-2%3A-Improving-Open-Language-Models-at-a-Size-Riviere-Pathak

[6] K. Ociepa, Ł. Flis, R. Kinas, K. Wróbel i A. Gwoździej, *Bielik 11B v2 Technical Report*, 2025. arXiv: 2505.02410 [cs.CL]. URL: https://arxiv.org/abs/2505.02410

[7] K. Ociepa, Ł. Flis, R. Kinas, K. Wróbel i A. Gwoździej, *Bielik v3 Small: Technical Report*, 2025. arXiv: 2505.02550 [cs.LG]. URL: https://arxiv.org/abs/2505.02550

[8] H. Renney, F. Trad, M. Mattarock i Z. Wood, "Cloud to Edge: Benchmarking LLM Inference On Hardware-Accelerated Single-Board Computers", *arXiv preprint arXiv:2604.24785*, 2026, Carnegie Mellon University; University of Exeter. URL: https://arxiv.org/abs/2604.24785

[9] G. Hinton, O. Vinyals i J. Dean, "Distilling the knowledge in a neural network", *arXiv preprint arXiv:1503.02531*, 2015, Foundational paper on Knowledge Distillation. URL: https://arxiv.org/abs/1503.02531

[10] L. Chen, M. Zaharia i J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance", *arXiv preprint arXiv:2305.05176*, 2023, Stanford University. URL: https://arxiv.org/abs/2305.05176

[11] Y. Leviathan, M. Kalman i Y. Matias, "Fast Inference from Transformers via Speculative Decoding", w: *Proceedings of the 40th International Conference on Machine Learning*, Google, PMLR, 2023, s. 19 274–19 286. URL: https://arxiv.org/abs/2211.17192

[12] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu i in., "The Prompt Report: A Systematic Survey of Prompting Techniques", *arXiv preprint arXiv:2406.06608*, 2024, University of Maryland. URL: https://arxiv.org/abs/2406.06608

[13] T. B. Brown i in., "Language Models are Few-Shot Learners", w: *Advances in Neural Information Processing Systems*, OpenAI, t. 33, 2020, s. 1877–1901. URL: https://arxiv.org/abs/2005.14165

[14] J. Wei i in., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", w: *Advances in Neural Information Processing Systems*, Google Research, t. 35, 2022, s. 24 824–24 837. URL: https://arxiv.org/abs/2201.11903

[15] P. Lewis i in., "Retrieval-augmented generation for knowledge-intensive nlp tasks", *Advances in Neural Information Processing Systems*, t. 33, s. 9459–9474, 2020, Key paper introducing RAG.

[16] T. Akiba, S. Sano, T. Yanase, T. Ohta i M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework", w: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Preferred Networks, 2019, s. 2623–2631. URL: https://arxiv.org/abs/1907.10902

[17] E. J. Hu i in., "LoRA: Low-Rank Adaptation of Large Language Models", w: *International Conference on Learning Representations*, Microsoft Research, 2022. URL: https://arxiv.org/abs/2106.09685

[18] T. Dettmers, A. Pagnoni, A. Holtzman i L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms", *Advances in Neural Information Processing Systems*, t. 36, 2024, Techniques for running LLMs on consumer hardware. URL: https://arxiv.org/abs/2305.14314

[19] T. Nguyen i in., "An Evaluation of LLMs Inference on Popular Single-board Computers", *arXiv preprint arXiv:2511.07425*, 2025, University of Technology Sydney. URL: https://arxiv.org/abs/2511.07425

[20] S. Kim i in., "SqueezeLLM: Dense-and-Sparse Quantization", w: *International Conference on Machine Learning*, UC Berkeley, 2023. URL: https://arxiv.org/abs/2306.07629

[21] P. Tummalapalli, S. Arayakandy, R. Pal i K. Kundan, "LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load", *arXiv preprint arXiv:2603.23640*, 2026. URL: https://arxiv.org/abs/2603.23640

[22] E. Frantar, S. Ashkboos, T. Hoefler i D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers", w: *International Conference on Learning Representations*, ETH Zurich, 2023. URL: https://arxiv.org/abs/2210.17323

[23] U. Kurt, "Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct", *arXiv preprint arXiv:2601.14277*, 2026. URL: https://arxiv.org/abs/2601.14277

[24] Silero Team, *Silero VAD: pre-trained enterprise-grade Voice Activity Detector*, https://github.com/snakers4/silero-vad, MIT License, 2021.

[25] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey i I. Sutskever, "Robust speech recognition via large-scale weak supervision", w: *International Conference on Machine Learning*, OpenAI Whisper Technical Report, PMLR, 2023, s. 28 492–28 518.

[26] ElevenLabs, *ElevenLabs: AI Voice Generator and Text-to-Speech Platform*, https://elevenlabs.io, Commercial TTS / voice AI platform, 2024.

[27] J. Kim, J. Kong i J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech", w: *International Conference on Machine Learning*, VITS: State-of-the-art Model for TTS, PMLR, 2021, s. 5530–5540.

[28] rany2, *edge-tts: Use Microsoft Edge's online text-to-speech service from Python*, `https://github.com/rany2/edge-tts`, Python wrapper for Microsoft Edge neural TTS voices, 2021.